Journal — Hardik Pajiala (1024030119)

Role: Person A — Frontend & UX (React) **Project:** FoodBridge — a surplus food redistribution matcher

Week 3 (11 – 15 October 2026)

What I did

Added the dietary-restriction constraint to the matching engine. I worked on this on Shikhar's machine, so it reached GitHub in a commit authored under his name. A recipient's declared restrictions are now a hard filter applied before any scoring, alongside capacity, radius and food preference, and the exclusion is returned in the matching explanation so an administrator can see why an organisation was passed over. Two tests cover it: a restricted food type is excluded, and an unrelated one still matches. The matcher suite went from 28 tests to 30.

Then found the gap that made it inert. The registration form had never sent `foodPreferences` or `dietaryRestrictions`. Both exist in the migration, the user repository, the auth service, the matcher client and the matcher's own model — and the form sent neither, so every organisation registered through the interface accepted every kind of food and declared no restriction. I added both as multi-selects, with a guard so that restricting a food type removes it from the accepted list.

Wrote the Software Requirements Specification. Seventeen pages, IEEE 830 structure: 55 functional and 22 non-functional requirements, business rules, constraints, glossary and open issues, with deferred items marked rather than omitted. I redrew the use case diagram from Shikhar's final version and added a table tracing all 26 use cases to the requirements that specify them — which found two, *Monitor Donations* and *Manage User Accounts*, with no API behind them at all. Recorded as FR-54 and FR-55, deferred.

Put three new metrics on the impact dashboard — rescue rate, first-offer acceptance and mean distance per allocation strategy. Two of those are items I recorded as blocked last week; the backend returns them now and the dashboard was ignoring all three.

Removed the N+1 request from the match inbox now that Person B has added the join, and **repaired the contract check** against the new, correct refusal of administrator self-registration.

Verified rather than assumed: 64 backend tests pass (42 last week), 30 matcher tests (28 last week), 25 contract checks with no failures, lint and build clean at desktop and phone widths. The dietary filter is proved end to end — an organisation that forbids dairy, offered a dairy donation beside it, comes back `eligible: false  dietary restriction excludes food type 'dairy'`.

What I learned

A field can exist at every layer and still be dead. `dietaryRestrictions` was in the schema, the repository, the service, the HTTP client and the matcher's model, and every test at every layer passed, because each layer handled correctly whatever it was given. The one place it was missing was the form — the only place a user can put a value in.

Comparing the two strategies on live data: greedy averaged 3.65 km over 8 offers, the Hungarian assignment 1.27 km over 3. The global assignment placed food closer but made fewer offers per cycle, so they are better on different measures — which is the question I raised last week, now with numbers.

Blocked / needs the team

- **The frontend is not in the main repository.** `main` holds the backend, the journals, the proposal and the report; there is no `frontend/` directory.
- FR-54 and FR-55 have no API behind them; we should agree to build or defer.
- The strategy comparison above comes from whichever cycles happened to run, so it is indicative, not a controlled experiment.

Next week

Get the frontend onto the main repository, agree the administrator oversight scope with Shikhar and Ayush, and test on a real handset.